

DEMOS: A BORDERLESS INTERCONNECTIVITY LAYER, ENABLING SEAMLESS INTEROPERABILITY AND ADVANCED FUNCTIONS ACROSS ALL CHAINS AND WEBS

Azhar Nurhussen | Cristiano Frassinetti | Jacob Ortiz Hansen
team@kynesys.xyz

Abstract

This paper presents a comprehensive analysis of DEMOS, a novel blockchain system that integrates various advanced concepts and technologies to create a secure, robust, efficient, scalable, and adaptable network. The design incorporates features such as Game Theory Incentives (GTI), Dynamic Validation Shard Size, Shard Generation with a Common Validator Selection Algorithm (CVSA), and the Secure Messaging Protocol (SeMP).

GTI is a unique incentivization strategy that promotes network stability, cooperation, and effective performance in a dynamic and decentralized gas blockchain. It utilizes game theory principles to encourage nodes to act in the best interest of the network. Nodes are assigned reputation scores based on their performance and contributions, which determine their likelihood of being selected as validators or proposers.

Dynamic Validation Shard Size enables the network to dynamically adjust the number of subdivisions (shards) based on transaction load and node distribution. This ensures optimal performance and resource utilization across varying network conditions. Load balancing techniques distribute the workload and nodes across shards, improving transaction processing speeds and reducing latency.

Shard Generation with CVSA ensures consistent and secure shard configuration by creating a shared, deterministic source of randomness. The process involves initiating an epoch, applying the CVSA, verifying and distributing the output, generating shards based on the output, and assigning nodes to these shards. This approach enhances security, scalability, and fairness in the network.

SeMP provides a secure messaging protocol for seamless communication and data exchange within the blockchain network. It incorporates passwordless authentication using private keys stored in hardware wallets, ensuring user identity integrity. End-to-end encryption and digital signatures protect message authenticity and confidentiality.

The paper also addresses other important aspects, including on-chain Web2 integration, crosschain logic execution, and on-chain storage. On-chain Web2 integration allows native access to Web2 resources within the blockchain network, leveraging cryptographic algorithms for authenticity and integrity verification. Crosschain logic execution enables scripting, execution, and consolidation of smart contracts and transactions across multiple networks, combining their strengths and mitigating limitations. On-chain storage optimizes file storage within the blockchain, addressing challenges related to file size, hashing, replication, and block inclusion.

Through the integration of these advanced concepts and technologies, the proposed blockchain system design demonstrates the potential for creating a highly secure, scalable, and efficient network suitable for a wide range of applications. It offers innovative solutions to incentivize node participation, adapt to varying network conditions, ensure secure communication, and provide efficient storage capabilities in a (highly) decentralized manner.

Introduction.....	4
DEMOS Core Structure and Principles.....	4
Nodes and Clients.....	4
Nodes and Validators.....	4
Nodes and Developers' Commitment to DEMOS.....	5
The Role of DEMOS.....	6
DEMOS SaaS Paradigm and Gas Fees.....	7
DEMOS Architecture & Consensus Mechanism.....	10
Decentralized Adaptive Architecture (DAA).....	10
The Global State Layer (GSL).....	12
Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT).....	14
Game Theory Incentives (GTI).....	16
Shard Generation and Validators Choice with a Common Validator Selection Algorithm (CVSA).....	17
Dynamic Validation Shard Size.....	18
Recap.....	20
On-chain Web2.....	21
Ensuring Integrity and Stability.....	21
Dealing with Dynamic Parameters.....	22
Crosschain Logic Execution.....	24
Design and Principles.....	24
Read, Write, and Crosschain Presence.....	25
Web2 and Cross-Web Logic.....	26
Crosschain Swapping.....	26
Design and Multiple Routes.....	27
DEMOS Firewall.....	28
Secure Messaging Protocol.....	29
Design and Principles.....	29
Passwordless Authentication.....	29
End-to-End Private Communication through DEMOS.....	30
On-chain Storage.....	31
Design and Rationale.....	31
The Size Dilemma.....	31
Hashing and Replication.....	32
Block Inclusion.....	32

Introduction

The following document presents **DEMOS**, an innovative decentralized network that interconnects and expands existing blockchain technology to address the limitations and challenges faced by traditional blockchain systems. DEMOS introduces novel concepts and mechanisms that aim to revolutionize the way decentralized networks operate and deliver trust, transparency, and scalability.

This yellow paper elucidates the key principles, design, and functionality of DEMOS, aiming to provide a comprehensive understanding of its inner workings. By exploring the decentralized nature of DEMOS and its nodal composition, we delve into the intricate mechanics that underpin this groundbreaking network.

DEMOS Core Structure and Principles

DEMOS represents a decentralized blockchain-like network, encompassing a distributed framework consisting of interconnected nodes.

Nodes and Clients

The term "node" refers to an entity that functions as a **container, operating on a standardized version of the DEMOS Node Software**, which is supplied by KyneSys Labs.

Given the **public** nature of DEMOS specifications, it is both feasible and strongly recommended to develop DEMOS clients capable of actively engaging in the DEMOS network. This approach fosters **client diversity** and enhances the overall functionality of the network.

Nodes and Validators

The DEMOS architecture is designed to enforce a validator role for every node within the DEMOS network. As a result, the DEMOS network achieves a decentralized L0-like blockchain structure, leveraging a particular consensus mechanism that heavily relies on the resilience and security principles inherent in the blockchain paradigm.

In order to ensure the commitment of validators and mitigate the risk of majority attacks, the operation of a node is contingent upon meeting specific prerequisites akin to staking (for further details, please refer to the next section). This approach aims to incentivize and reward honest and

productive participants within the network while penalizing those who engage in malicious activities or exhibit inadequate performance.

The Role of DEMOS

In order to grasp the essence of DEMOS' purpose and its potential capabilities, it is imperative to gain a comprehensive understanding of the network's fundamental role, delving into the intricacies of its multifaceted offerings.

DEMOS, functioning as a network-agnostic crosschain infrastructure, has been meticulously architected to facilitate seamless communication, interaction, and mediation among the realms of web2, web3, and the myriad of existing blockchains.

While DEMOS acknowledges and recognizes the inherent value and benefits offered by diverse L1, L2, and L3 blockchains currently in existence, its primary objective is not to undermine their utility. Rather, DEMOS assumes the role of **decentralizing the blockchains themselves**, empowering users and developers to harness the optimal attributes embedded within each distinct chain.

As explained in this paper, all the features that DEMOS provides are made with the specific goal of **enhancing decentralization without reinventing the wheel**. Each technology and blockchain possesses a unique set of strengths and weaknesses. It is within this context that DEMOS emerges as a solution, enabling seamless interoperability among different technologies. Moreover, DEMOS enriches the entire ecosystem by incorporating its own distinctive features, thereby fostering a cohesive and **unified environment**.

DEMOS SaaS Paradigm and Gas Fees

When it comes to the world of decentralized applications (dApps), DEMOS emerges as an avant-garde Software as a Service (**SaaS**) platform. DEMOS revolutionizes the traditional paradigm by shifting the financial responsibility of the service call from the user to the developer while still **allowing** the developers to have their users to cover them, creating an ecosystem that is both **equitable** and **user-centered**. While DEMOS is not free—except for a select few basic features such as messaging, certain crosschain capabilities, and storage capacities up to a particular size—its unique structure makes it a **highly attractive choice** for both developers and users alike.

One of the key aspects of DEMOS is its gas system, a pivotal component for executing operations within the network. In the DEMOS system, the Remote Procedure Call (RPC), which functions behind the dApp, calculates the gas, or the computational effort, required for executing a particular operation. This calculation isn't made **haphazardly**—it accounts for the cost of the call, the gas fees of each participating chain, and the desired profit margin of the developer. This ingenious architecture ensures **transparency** while also allowing developers to **strategize** their investments for optimized returns.

Let's delve a little deeper with an illustration ([Figure 1](#)). Imagine a dApp that offers **crosschain swaps using the DEMOS network**. To facilitate this operation, the dApp needs to call upon a DEMOS RPC. At this juncture, the developer has two potential routes to take.

The first option allows the developer to run its own node by staking resources in the DEMOS network. In this scenario, the developer **does not need to pay any external fees**, making it a financially appealing choice for those with sufficient resources. The dApp calculates the gas fee by **considering the gas fees across the different chains involved in the transaction plus the profit** they aim to generate. This ensures that the cost of operation is directly proportional to the resources consumed and the profit the developer wishes to generate, creating a direct link between investment and gain.

The second option caters to developers who, for various reasons, opt not to run their own nodes. Instead, they can leverage a **public RPC**, paying the required fees as determined by the public RPC. Here, **the gas fee calculation includes the RPC fee**—which is calculated based on the gas costs across the various chains and the desired profit—**plus the profit** that the developer themselves want to achieve.

What makes DEMOS's structure so enticing is its inherent flexibility, which can accommodate developers with diverse capabilities and resources. Developers with the resources and capacity to **run and manage their nodes** can sidestep the fees associated with public RPCs, effectively

reducing their operational costs. This opens up the possibility of offering **more competitive prices** to their users, potentially drawing in a larger user base and boosting their market presence.

Conversely, developers without the resources to run their nodes can still actively participate in the ecosystem by utilizing **public RPCs**. This inclusive model ensures that the DEMOS network remains **accessible** and **equitable**, fostering greater diversity and innovation within the ecosystem.

In conclusion, DEMOS's cutting-edge SaaS platform presents an exciting new frontier for developers in the dApp space. Its **innovative handling of gas fees**, where the developer shoulders the cost and present the user with a **simple, fair way** of covering the fees, empowers greater participation from diverse developers. This, in turn, enriches the ecosystem, paving the way for a more vibrant, dynamic and user-focused blockchain environment.

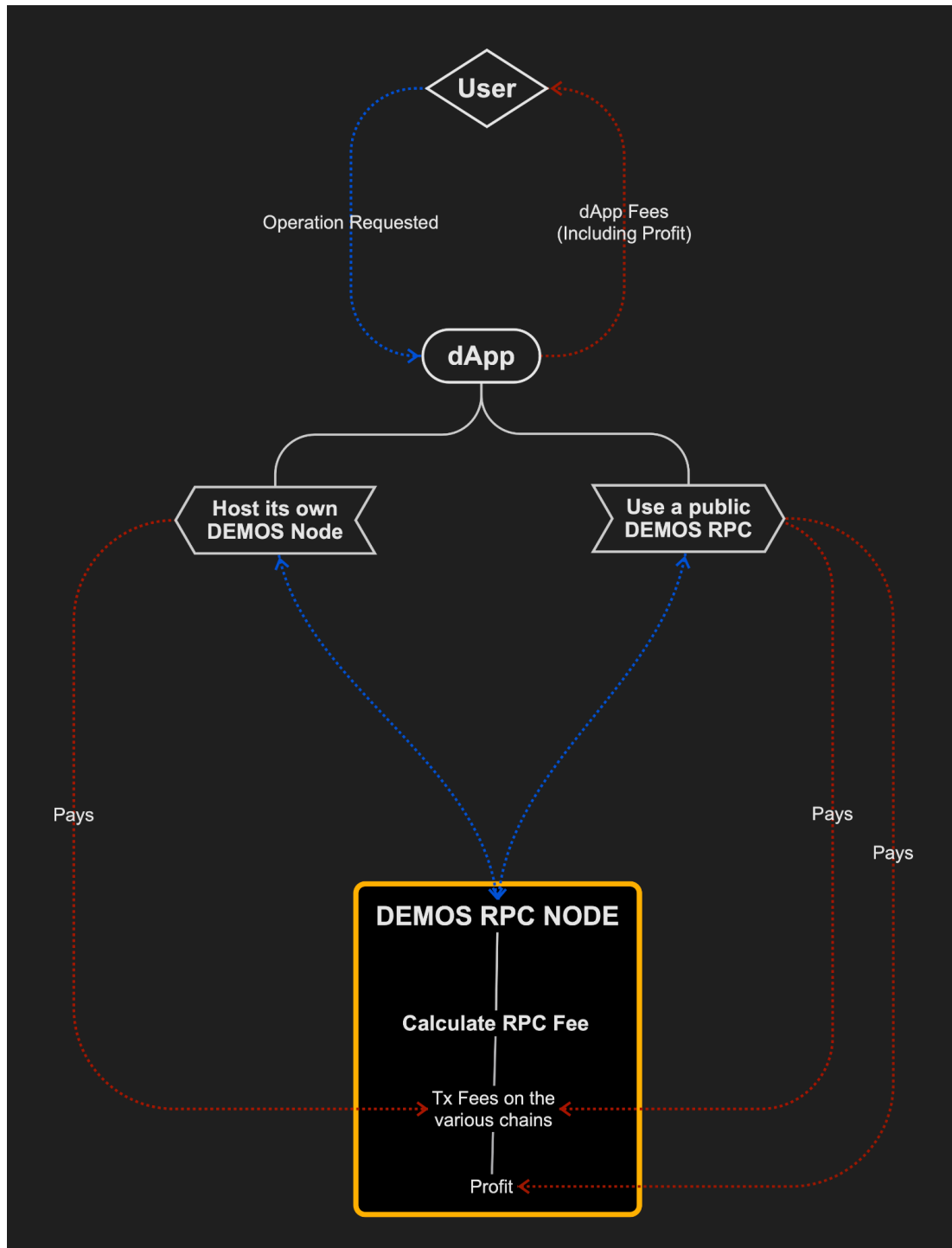


Figure 1 - DEMOS transaction fees workflow

DEMOS Architecture & Consensus Mechanism

Decentralized Adaptive Architecture (DAA)

Overview

Decentralized Adaptive Architecture (DAA) is an **innovative architecture model** designed to achieve exceptional efficiency, scalability, and performance in decentralized systems while operating as a **parachain infrastructure**. DAA leverages dynamic system adaptation, game theory, and representative sharding technology to ensure robust security.

Mechanism

1. Global State Layer (GSL)

To enhance scalability and efficiency, DAA implements the Global State Layer (GSL) paradigm. This mechanism involves processing transactions in the most decentralized and collaborative way, so that it is easy, fast and efficient to find and scrap invalid or malicious payloads. More, the Global State Layer ensures consistency across shards while allowing them to operate independently.

2. Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT)

DAA employs a two-layer byzantine consensus mechanism, combining Proof-of-Representation (PoR) with Byzantine Fault Tolerance (BFT) for updating the Global State Layer. Nodes are selected to form a Representative Shard for the validation process based on the PoR algorithm that ensures a deterministic non-tamperable selection method. This incentivizes stability and efficiency. Within the Representative Shard, a Byzantine Fault Tolerant consensus protocol is implemented, guaranteeing robustness, security, and scalability without burdening the entire chain.

3. Game Theory Incentives (GTI)

To help ensuring profitability, competitiveness and commitment, DAA incentivizes nodes via **a free market fee system**. Using game theory principles, nodes are able to provide their services for a certain fee over the base chain fee, so that each node competes with the others to offer the most efficient and cheapest service possible while maintaining the profitability.

4. Dynamic Validation Shard Size

DAA continuously adapts to varying transaction volumes by dynamically adjusting the number of participants to the block proposing consensus phase. This adaptability is achieved through a randomness layer powered by a Common Validator Selection Algorithm, ensuring the generation of a dynamic and consistent Representative Shard.

Advantages

1. **Enhanced Scalability:** Through the implementation of Common Validator Selection Algorithm, the Decentralized Adaptive Architecture (DAA) system achieves superior processing capabilities by concurrently handling a multitude of transactions within a deterministic representation of all the nodes. This substantially augments the network's overall capacity, enabling seamless scaling.
2. **Robust Security:** The DAA system incorporates the classic Byzantine Fault Tolerant (BFT) consensus technique in conjunction with the Proof-of-Representation model to establish a formidable security framework that effectively mitigates a wide range of potential attack vectors. This amalgamation of methodologies ensures the integrity and resilience of the system against malicious activities.
3. **Heightened Efficiency:** DAA demonstrates remarkable efficiency by dynamically adjusting the proposers count in response to the network's load. Leveraging the power of BFT, independent cross-node validation is facilitated, leading to optimized processing speeds and resource utilization. The system operates at peak efficiency, bolstering overall performance.
4. **Sustained Decentralization:** By integrating Game Theory Incentives, DAA upholds the fundamental principles of decentralization within the blockchain ecosystem expanding it to include a fair and competitive fee system. This mechanism incentivizes nodes to collaborate in the best interest of the entire network, fostering a robust and resilient decentralized environment. The preservation of decentralization ensures equitable participation and governance within the DAA system.

The Global State Layer (GSL)

GSL is a mechanism that **allows transactions to be executed** with the possibility of rolling back, examining and verifying their content even before a block is emitted ([Figure 2](#)).

Mechanism

1. A transaction reach an RPC and are validated for identity and integrity. The signature and the hashes are examined and confirmed, then the transaction is sent to the GSL Preprocessor..
2. Global State Layer (GSL) Preprocessor: The GSL functions as a comprehensive ledger that maintains the state of the entire blockchain network. The Preprocessor examine the transaction and build a delta (a sorted registry of changes) to the Global State Layer without modifying it directly.
3. Each subsequent transaction follows the same process as above. Only when a block need to be forged from the mempool, all the GSL Preprocessor registry items are executed one by one in the right order on a copy of the GSL.
4. If a transaction produces an invalid result in the GSL, is rejected or reverted. Otherwise, the GSL gets permanently modified once the block is emitted by the consensus mechanism.

Key Features and Solutions

1. Predictive Transaction Validation

Thanks to the local GSL change registry, each node is able to predict with a reasonable precision if a transaction will fail or will succeed. Even before the next block is emitted, many elements and mechanisms allows a transaction to be simulated with efficiency and rapidity.

2. Resource Friendly Rollbacks

For the same reason, as the GSL change registry is not enforced until the block is emitted, it is very easy to track and roll back invalid, malicious or malformed transactions. Even if a transaction makes it until the GSL change registry, once the consensus validator parse it and flag it as invalid there is no need to do anything else than scrapping the transaction from the registry.

Advantages

1. Scalability: GSL (Global State Ledger) leverages parallel processing of transactions within individual nodes and employs efficient global state management techniques to significantly enhance the network's capacity.

2. Efficiency: The employed strategy effectively minimizes cross-nodes communication overhead and optimizes GSL updates, thereby achieving a notably high level of operational efficiency.
3. Consistency: By diligently maintaining a synchronized state of the blockchain across all nodes, GSL ensures a unanimous agreement among all network nodes regarding the current network state.
4. Enhanced Interoperability: The incorporation of GSL in the system facilitates improved interoperability among different nodes within the blockchain network. Consequently, any node gains the ability to seamlessly access the state of any address present across the entire network.

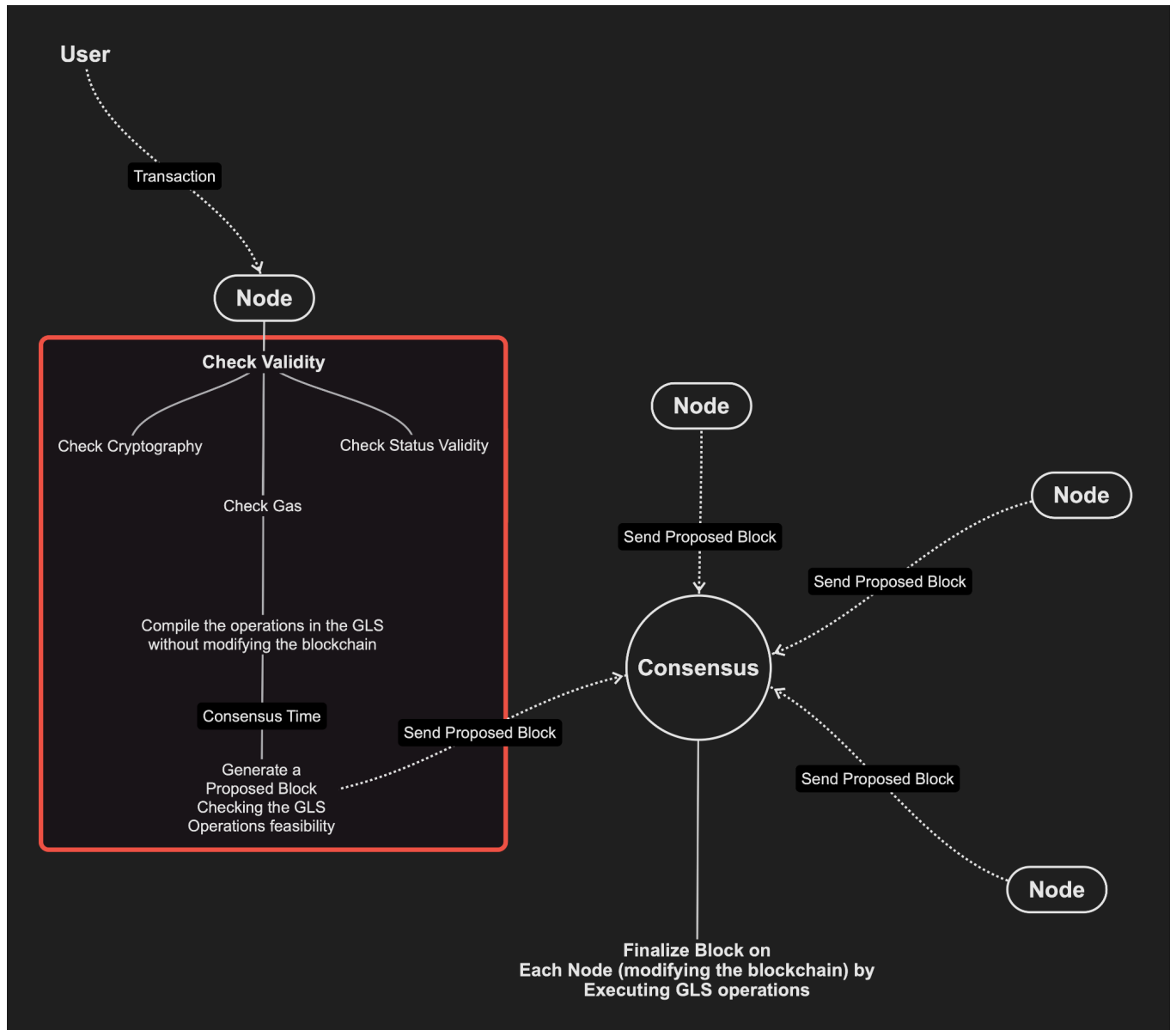


Figure 2 - Transactions Flow in the DEMOS Network

Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT)

Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT) is an advanced consensus mechanism that operates at a heightened level of technical sophistication, effectively ensuring network stability, efficiency, and security within a blockchain system. It accomplishes this through the integration of a Byzantine Fault Tolerance (BFT) consensus mechanism within a shard representing the entire validators network, **selected** with the utilization of Proof-of-Representation (PoR) for the proposal of blocks to the Global State Layer (GSL).

This shard is called the Representative Shard and is composed of the result of an algorithm that, using the deterministic property of the transactions in the mempool, ensures the same group of validators are chosen by all the legitimate nodes without any need of data transfer.

The Two Consensus Tiers

1. Byzantine Fault Tolerance (BFT) within a Shard

The PoR-BFT system leverages the Byzantine Fault Tolerance consensus mechanism. BFT enables rapid validation and agreement of transactions within the shard, even when confronted with the presence of malicious or faulty nodes. By mandating the agreement of a supermajority (typically $2/3+1$) of nodes prior to appending a new block to the shard's blockchain, BFT ensures efficient operation and robust security within a group of validators..

2. Proof-of-Representation (PoR) for a Representative Shard

To elect the Representative Shard, the PoR-BFT system employs the PoR mechanism. Validators selected for this process are chosen based on deterministic values contained in the mempool and in the calculated status changes for the block to forge, which will always be identical for all the nodes that act on the same, untampered and verified data. This way, malicious nodes or malfunctioning nodes are automatically excluded from the consensus selection without any network overhead.

Advantages

1. Efficiency: The utilization of a Byzantine Fault Tolerant (BFT) consensus mechanism within a Representative Shard facilitates rapid transaction processing and streamlined intra-shard operations.
2. Augmented Security: The Proof of Representation (PoR) consensus mechanism significantly strengthens security measures during the validators election process to the next consensus round. By selecting nodes using a deterministic approach based on data integrity, PoR effectively

mitigates the risks associated with malicious activities, fostering heightened integrity and efficiency within the blockchain system.

3. Scalability: The PoR-BFT approach offers an advanced and scalable solution that can effectively manage a substantial volume of transactions. This approach ensures consistency and reliability within the GSL, even in the face of high transaction loads, thereby enabling the system to seamlessly handle increased demands without compromising performance. This is done by changing the size of the Representative Shard to facilitate computation and data transfer even in case of massive traffic.

4. Stability: The PoR mechanism promotes stability within the blockchain network by avoiding data overhead and by incentivizing long-term participation from network nodes. By dynamically select a Representative Shard for the whole blockchain, PoR fosters an environment of reliability and predictability, ensuring the overall stability of the blockchain system.

Game Theory Incentives (GTI)

Game Theory Incentives (GTI) is a unique incentivization strategy that promotes network stability, cooperation, and effective performance in an **extremely low gas fee** blockchain system. Rather than relying on direct financial rewards, GTI utilizes principles from game theory to encourage nodes to act in the best interest of the network.

Design and Mechanism

1. Dynamic Gas Fees

Each node within the system calculates the base network fee to apply to a transaction using a GLS shared state and a common algorithm. The DEMOS network expects this fee to be compliant and will require that amount to be paid fully. Factors as network load, complexity and size all concur to define how much is this fee.

2. User Agnostic Profit Flexibility

On top of the base DEMOS fee, nodes are able to request an additional fee which quantity is arbitrarily decided by the node maintainer. This way, node operators can define a profit that is independent from the user and that will make running a node profitable.

3. Competitive Landscapes

This possibility enables a competitive landscape where each node has to find the right balance between offering competitive fees and having enough profit to be efficient and performant. As per the game theory, a landscape like this brings to the decentralization of the actual gas fee based on the node attractiveness for a developer or an user.

Advantages

1. Encourages Active Participation: GTI encourages nodes to maintain active and reliable participation in the network, as this increases their chances of being selected as RPC by developers and users.

2. Enhances Security: By rewarding efficient and transparent nodes, GTI helps maintain the integrity and security of the network.

3. Promotes Fairness: The natural selection process ensures all nodes, regardless of their elderness, have a chance of being an RPC.

4. Improves Network Performance: GTI encourages cooperation among nodes, leading to improve overall network performance.

Shard Generation and Validators Choice with a Common Validator Selection Algorithm (CVSA)

The process of shard generation using a Common Validator Selection Algorithm (CVSA) involves creating a shared, deterministic source of **randomness** to ensure all nodes generate the same shard configuration given the same network state ([Figure 3](#)).

Phase 1 - Initiate Epoch

At the commencement of each epoch, the blockchain system initiates the CVSA process. This process commences by selecting an initial input, commonly denoted as the "seed." The determination of this seed can be derived from the hash of the preceding epoch's last block or any other deterministic value that is shared across the network (like the estimated result of a legit proposed block).

Phase 2 - Apply CVSA

The selected seed is then input into a pseudo random number generator to generate a CVSA. The CVSA will be identical for those nodes compliant with the blockchain mechanism while will be different for malicious or malfunctioning nodes. The output remains impervious to prediction until the computation is finalized and needs to be demonstrated by cryptography, ensuring security and unpredictability.

Phase 3 - Verification and Distribution

Once the CVSA has generated the output, all other nodes will possess the same information without need of transmission. The inherent property of rapid verification in the CVSA guarantees simultaneous propagation of the output to all nodes.

Phase 4 - Shard Generation

Each node in the network uses this output as the seed for shard generation and validation assignment. Since the seed is identical for all nodes, the shard generation process results in the same set of shards across the entire network. The nodes are then assigned to these shards based on the deterministic process using the same seed, ensuring uniform distribution and balanced workload.

Phase 5 - Begin Epoch Operations

With the shards generated and nodes assigned, the blockchain network proceeds with its operations within the new epoch, processing transactions within each shard, and committing states to the Global State Layer.

The utilization of a CVSA within the randomness layer provides a secure, verifiable source of randomness, enabling consistent shard generation across the network, enhancing security, and ensuring scalability.

Dynamic Validation Shard Size

Dynamic Validation Shard Size entails the implementation of a sophisticated approach wherein the number of actors chosen between the nodes network to operate the consensus, known as Representative Shard, within the blockchain network can be dynamically adjusted in response to variations in transaction load and node distribution. This adaptability grants the blockchain network the ability to uphold optimal performance levels across a **diverse range of conditions**, spanning from scenarios of low to high transactional demand.

Design and Mechanism

1. Choosing a Representative Shard: each block time, DEMOS blockchain utilizes a deterministic algorithm that returns the same exact result on all the nodes that have the same data in their mempool and in their blockchain. This way, all the nodes are able to instantly know who the participants are without any communication needed.
2. Parameters: to create the Representative Shard, parameters based also on the chain load, throughput and specific conditions are enforced to determine the size of the Representative Shard.

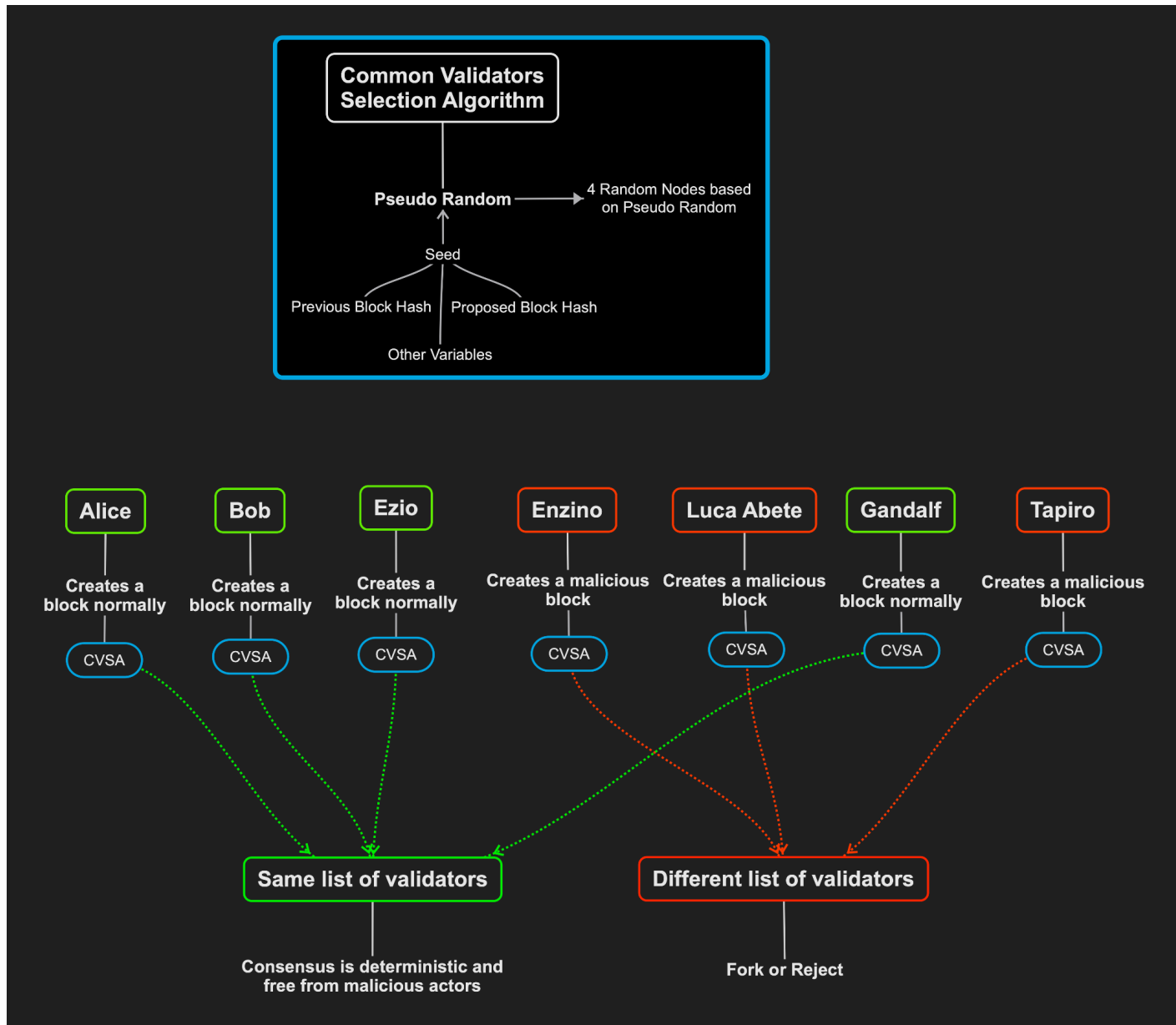


Figure 3 - Common Validators Selection Algorithm

Recap

The newly proposed blockchain consensus mechanism presents a **robust, scalable, and innovative** solution, integrating features of Global State Layer (GSL), Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT), Game Theory Incentives (GTI), Dynamic Validation Shard Size, and a secure Randomness Layer powered by a Common Validator Selection Algorithm (CVSA).

Through ACS-GSL, the system **efficiently** manages **transaction** processing and **data storage** across various shards, enhancing scalability while ensuring a **consistent** view of the global state.

The PoR-BFT consensus model integrates a Byzantine Fault Tolerance consensus within each shard with a Proof-of-Representation mechanism for global block proposals, optimizing both intra-shard and inter-shard operations.

The GTI framework encourages **active, reliable participation and cooperative behavior** among nodes, fostering network stability and security while ensuring **profitability and fairness**.

Dynamic Validation Shard Size enhances the system's **adaptability**, ensuring optimal **performance** and resource utilization regardless of network load. This, combined with the integration of a CVSA-powered Randomness Layer, ensures secure, verifiable, and consistent shard generation, further contributing to the network's robustness and security. This innovative approach to generating shared randomness enables the system to maintain uniformity and fairness in the node distribution and workload allocation, further enhancing the network's scalability.

This novel blockchain system design showcases the potential of integrating various advanced concepts and technologies in building a blockchain network that is not only secure, robust, and efficient but also highly scalable and adaptable to varying network Conditions.

On-chain Web2

The establishment of a seamless linkage between web2 and web3 serves as a crucial solution to a significant impediment **hindering** the emergence and flourishing of many projects and technologies.

DEMOS Network leverages its **decentralized** structure and battle-proven **cryptographic** algorithms to provide seamless, native, and trustless access to web2 resources from within blockchains.

Design and Mechanism

At the core of web2 integration, there is a specific endpoint available to dApps and clients that allows actors to ask for a specific resource such as an API or a GraphQL request. Once the RPC receives a correctly compiled request on the web2 endpoint, it first **validates** the request by checking the hash of the request and its **signature** to ensure authenticity and integrity.

Then, it retrieves the requested data producing a pending status response that contains the resource, its hash, and the signature of the node. A specific request to peers is then propagated through the network, containing the pending status response. This enables them to engage in a process of **attesting or invalidating** the query's outcome while asynchronously filling in the pending response. Once a sufficient number of witnesses affirm the same result, the RPC is able to send back the response to the client or dApp providing **proof of integrity and validity** secured by the DEMOS network.

The employed hashing algorithm is **SHA256**, and the signature method utilized is **ed25519**. This empowers the on-chain retrieval and provision of web2 data without reliance on external oracles or centralized solutions.

While this mechanism is secure enough to be trusted, a couple of **control mechanisms** are in place to also ensure performance, error handling, and the resolution of specific exceptions that may occur during the integration of web2 and web3 ([Figure 4](#)).

Ensuring Integrity and Stability

To mitigate the **unpredictable nature of web2** data retrieval and prevent potential clogging and security concerns, a **robust** system must be put in place. This system employs a multitude of techniques such as request throttling, End-To-End validation, and replication verification to ensure optimal performance and security.

Request Throttling

To prevent **overloading** low-performance endpoints and to ensure that responses are received within an acceptable timeframe, request throttling is employed. This technique involves limiting the number of requests that can be made within a specific timeframe. By implementing request

throttling, web2 systems effectively manage their resources, preventing congestion and optimizing performance. Request throttling is implemented using various techniques built on two main paradigms: **rate limiters**, which restrict the number of requests based on a predetermined rate, and **timeframe restriction**, which restricts the timeframe in which a request can be retrieved, attested, and sent back.

End-to-End Validation

End-to-End (E2E) validation constitutes a crucial technique for guaranteeing the authenticity of web2 data retrieval. This process involves thorough verification of the identities of all parts involved before any data exchange takes place. By enforcing E2E validation, web2 systems effectively counter potential security threats, such as man-in-the-middle attacks and unauthorized access. On the web3 side of the request, E2E validation is implemented using **on-chain signature-based** identity verification, while **digital certificates** are employed on the web2 side to ensure the integrity of the request and response flow

Replication Verification

Replication verification serves as a fundamental technique for safeguarding the **integrity** of web2 data retrieval. This process entails verifying that the provided data has been **replicated by a sufficient number of nodes** to establish its validity. By employing replication verification, web2 systems effectively mitigate potential security risks, including tampering and unauthorized modifications.

The application of replication verification involves utilizing a **Byzantine Fault Tolerant (BFT)** consensus algorithm within a **shard of nodes** responsible for attesting or rejecting the same request. Through the use of **signed hashes**, which generate a unique digital fingerprint of the data, all nodes within the shard reach a consensus on the data's validity, enabling multiple nodes to independently verify its integrity.

Dealing with Dynamic Parameters

In order to mitigate the occurrence of **broken responses**, particularly those caused by dynamic parameters, an advanced methodology is employed within the on-chain web2 environment.

This methodology revolves around adopting a meticulously tailored approach to requests, which prioritizes the utilization of hashed parameters while disregarding **dynamic counterparts**. To achieve this, clients and decentralized applications (dApps) are strongly advised to precisely specify the parameter, field, or variable they require, ensuring that only the designated element is utilized for the purpose of result verification.

Even when confronted with dynamic fields, such as **timestamps**, which possess the potential to differ across nodes involved in attesting the result, this approach ensures utmost accuracy by strictly confining the attestation process solely to the elements explicitly requested.

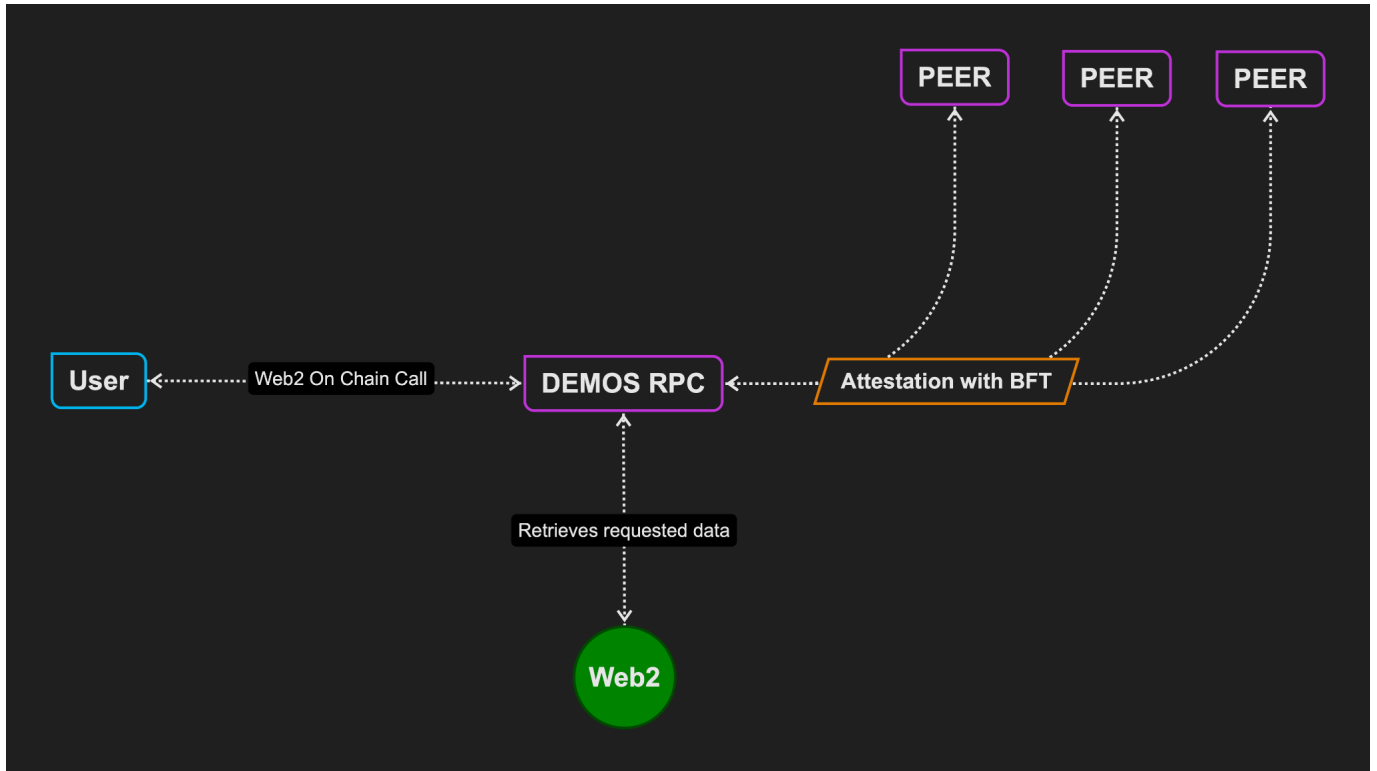


Figure 4 - Web2 On Chain Flow

Crosschain Logic Execution

The concept of Crosschain Execution is the integration of diverse domains, combining attributes like security and speed by empowering developers to script, execute, and consolidate smart contracts and transactions across **multiple networks**.

By harnessing the strengths of individual networks and mitigating their respective limitations, Crosschain Execution establishes a more efficient and secure system as a whole. For instance, developers can leverage the speed and efficiency of one network while compensating for its lack of robust security features by **utilizing another network** that offers enhanced security capabilities.

This approach offers flexibility and adaptability, enabling developers to select the most suitable network for each specific task, unrestricted by a single network. It fosters a decentralized ecosystem, devoid of any dominant controlling entity, by creating an **interconnected layer** with DEMOS, facilitating seamless data and functional transfer between networks. This interconnectedness allows for efficient resource exchange and utilization, promoting a cohesive and synergistic ecosystem.

Design and Principles

Crosschain Execution refers to a revolutionary design principle that facilitates efficient and rapid interaction between different blockchains. This advanced concept is particularly advantageous in scenarios where smart contracts or a collection thereof necessitate a substantial computational workload. For instance, consider a dApp developer who desires to maintain the security of a **gas-intensive chain** like Ethereum while **capitalizing on the lower gas costs** associated with a less robust chain such as Binance Smart Chain (BSC).

The introduction of Crosschain Execution has made it possible to execute portions of a smart contract on BSC, while the ultimate outcome is validated and finalized on Ethereum. By adopting this approach, developers can harness the swiftness and efficiency of BSC while simultaneously upholding the security and functionality offered by Ethereum. Furthermore, Crosschain Execution empowers developers to create intricate dApps that exploit the unique strengths of multiple blockchain networks, **without confining themselves to a single chain**.

To enable the implementation of Crosschain Execution, DEMOS has developed user-friendly standardized endpoints and **software development kit (SDK) functions**. These resources facilitate crosschain execution for both clients and developers. The platform's unified

architecture adheres to the principle of interoperability, ensuring **seamless collaboration** among diverse blockchain networks.

Each supported chain encapsulates its available methods, such as establishing a connection with a provider or retrieving balances, within a standardized object that exposes identical methods across all supported chains. This standardized approach simplifies the execution of transactions across different blockchain networks, as developers can utilize a single method, such as "Connect," which remains applicable to every supported chain. Moreover, the platform's SDK functions furnish developers with a straightforward and intuitive interface to interact with the blockchain network, thereby **streamlining the process of building decentralized applications**.

The SDK **undergoes continuous maintenance and updates** to accommodate the expanding range of supported blockchains. Importantly, these enhancements are implemented without necessitating developers and users to make significant changes to their existing code, thereby **minimizing the learning curve** while preserving flexibility and simplicity.

Similarly, transaction execution adheres to the same underlying principles, bolstered by an additional layer of security that mandates authentication of the requester before executing any write-like operation.

Read, Write, and Crosschain Presence

To enable interoperability across various blockchains, DEMOS employs a comprehensive framework that supports two distinct types of operations within the domain of Crosschain Execution.

The **Read** operations entail retrieving data from target networks without requiring explicit authentication, except for specific scenarios. For instance, examining the account balance on Solana **does not necessitate** possessing a Solana account, and this principle holds true for nearly all read operations across diverse networks. This operational category proves immensely valuable for swiftly obtaining data without the need to engage in time-consuming processes like handshake establishment, authentication, and data propagation associated with on-chain modifications.

The **Write** operations demand meticulous and trusted authentication on the target network to ensure secure execution. Since these operations involve altering data on-chain, **utmost caution, efficiency, and discretion** must be exercised to ensure data accuracy and identity integrity.

By leveraging **client-side signatures** when possible, DEMOS effectively serves as a virtual RPC mechanism for the target network, while concurrently exposing the aforementioned universal DEMOS methods. Consequently, DEMOS **does not require the user's private key** or sensitive

information, instead acting solely on behalf of the requester for that specific request, utilizing the specific content provided by the user.

On the other hand, certain operations may necessitate a more direct approach wherein relayed transactions relying solely on signatures prove insufficient. In such scenarios, DEMOS assumes an **intermediary role**, executing logic directly on a blockchain while soliciting additional requirements from the requester, such as gas fees or specific information.

In both cases, DEMOS is able to select the **most efficient route** and deliver an intuitive, user-friendly, and privacy-focused solution that remains agnostic to the underlying blockchain, benefiting developers, users, and clients alike.

Web2 and Cross-Web Logic

As DEMOS is built around different features that **can and will interact** with each other, one of the derived functionalities, known as Cross-web Execution, is a **combination of Crosschain Execution and On-chain Web2**, with its technical explanation being the sum of these constituent elements. It is necessary to highlight this **synergistic** feature, which may not be immediately evident.

By employing the established methodologies for on-chain retrieval of web2 resources with assured integrity and security, alongside seamless and secure crosschain smart contract execution, the **convergence of these approaches** gives rise to unparalleled capability.

For instance, a dApp can incorporate **smart contract logic** deployed on an EVM network while leveraging attested **web2 data** as input. Conversely, developers have the flexibility to feed the results of a smart contract into a web2 API, such as an **AI endpoint**.

The potential applications are virtually limitless, contingent upon the performance characteristics of the involved endpoints. Consequently, the resulting data can be **once again utilized** in a similar manner across other networks, endpoints, or technological frameworks.

Crosschain Swapping

Crosschain Swapping, as an implementation of crosschain execution, constitutes a **distinctive** use case that necessitates a dedicated exposition of its fundamental mechanics.

To facilitate a comprehensive understanding of crosschain swapping's underlying technical principles, it is beneficial to provide a brief explication of this feature. Crosschain Swapping entails the seamless, rapid, and efficient execution of **direct swaps** between disparate blockchain

networks. For illustrative purposes, we shall consider a practical scenario where **an ERC20 token is acquired using a Stablecoin on the Solana blockchain**.

Design and Multiple Routes

During the development of crosschain swapping, it became evident that a single method could not consistently deliver the optimal combination of efficiency, speed, and security.

Due to varying conditions on both sides of the swap operation and within the DEMOS network itself, DEMOS must possess the intelligence to **select the best overall route** from a set of possible options.

To address this requirement, we have devised **two primary scenarios** in which DEMOS can make decisions regarding the chosen path.

1. Full Liquidity Availability:

In the ideal situation, DEMOS will possess **sufficient liquidity on both sides** of the swap, enabling it to promptly provide liquidity after validating the swap request. In this case, DEMOS will either distribute the requested token directly to the buyer's address or, if necessary, exchange the available liquidity for the requested token before distribution. This method, which is the fastest and **preferred** approach, will be employed whenever the Full Liquidity Availability condition is met.

2. Limited Liquidity Availability:

When DEMOS lacks the necessary liquidity on both sides of the swap, it acts as a **bridge and ramps aggregator** with enhanced security measures. This scenario is expected to occur during the initial stages of DEMOS implementation.

While **maintaining the same interface and syntax** for developers and users, internally DEMOS scans and selects from a list of **bridges** (on-chain) and **ramps** (off-chain **via web2 endpoints**) to identify the most efficient and secure route. This eliminates the need for the requester to manually select a bridge or ramp, thereby saving time and minimizing the exposure of funds.

In addition to this efficient selection process, DEMOS incorporates a **firewall layer** that continuously and heuristically detects security issues on bridges and ramps as quickly as possible. The mechanism employed by the firewall operates in diverse ways, which will be explained in the following paragraph.

DEMOS Firewall

In the context of the Limited Liquidity Available scenario, as previously mentioned, DEMOS employs a sophisticated mechanism to identify and select the optimal bridge or ramp for providing a seamless, efficient, and expeditious experience to the requestor.

As widely acknowledged, **bridges represent a prominent target** for malicious actors seeking to exploit vulnerabilities and steal funds. To counteract this threat, DEMOS incorporates a firewall layer.

Constant Auditing

The DEMOS network employs a perpetual and automated process to audit the smart contracts and components of ramps and bridges, meticulously scrutinizing them for common vulnerabilities, exploits, and attacks. Despite variations in code implementation, it is noteworthy that a majority of bridge exploits stem from **similar types of vulnerabilities**. DEMOS vigilantly **monitors** the security posture of the platform to proactively thwart the utilization of insecure pathways, all while remaining imperceptible to the end user.

Immediate Response Propagation

In instances where a bridge or ramp falls victim to exploitation, the attackers' primary source of gain does not solely derive from the initial attack. The most substantial harm is inflicted upon users who, oblivious to the service's compromise, **continue utilizing it for prolonged periods**, sometimes spanning hours or even days, until they become aware of the situation.

Within this realm, DEMOS assumes the role of an all-encompassing, automated monitoring service. Even in cases where an exploit employs ingenious tactics that evade immediate detection, DEMOS leverages heuristic analysis of transactions, news reports, and official statements to swiftly identify the issue and **impede vulnerable routes for all DEMOS users**, without them noticing it.

Leveraging the aforementioned techniques, the DEMOS Firewall manifests as an **advanced system** that integrates heuristics, analytics, and machine learning, effectively safeguarding users, even when operating as an intelligent aggregator of bridges and ramps.

Secure Messaging Protocol

Design and Principles

The DEMOS network offers an advanced functionality known as the Secure Messaging Protocol (**SeMP**), which grants users the capability to engage in seamless communication and data exchange within a **trustless**, **uncensorable**, and **highly efficient** framework. While numerous on-chain messengers have been proposed and off-chain end-to-end encrypted messengers are widely utilized by millions of individuals daily, an innovative blockchain-based universal messenger that leverages the strengths of these **two web technologies** has yet to be witnessed.

The Secure Messaging Protocol implemented in DEMOS does not depend on interchain operations or on-chain data from web2 sources. Rather, it is an **intrinsic feature** exclusive to the DEMOS ecosystem, harnessing the underlying infrastructure of the network

Passwordless Authentication

The primary and immediate concern in the realm of personal social applications, encompassing social networks and messenger apps alike, pertains to the **effective authentication** of users and the prevention of malicious actors attempting to impersonate legitimate users.

While numerous systems rely on **password-based** and **two-factor authentication (2FA)**, which generally proves effective, it is unwise to presume that users will consistently uphold their responsibilities in maintaining system security. Any slight negligence on the user end can become an exploitable vulnerability, a reality we encounter daily and, regrettably, accept as **inevitable**.

SeMP addresses this issue by **minimizing the reliance on user actions**. Once a user can successfully sign a login token with their private key, which can reside within a hardware wallet, for instance, authentication is established. The act of stealing a private key presents a considerably more intrusive and intricate endeavor compared to stealing a password.

If a password is stored **locally**, extracting it becomes effortless when accompanied by appropriate software bundled with any program executed on the victim's computer. If the password is entered each time, the security can be undermined by a **keylogger**. Similarly, 2FA methods can be **circumvented** through the same means or via social engineering tactics.

On the other hand, when a private key is utilized within a wallet, it is typically encrypted, rendering it **indecipherable** to malicious users. Furthermore, there is **no need for constant**

re-entry of the key. The proliferation of hardware wallets and offline signing capabilities facilitates the segregation of sensitive information from connected systems.

End-to-End Private Communication through DEMOS

Once a client establishes a connection via its cryptographic DEMOS credentials, all subsequent messages exchanged between the client and another party, such as a friend, undergo a two-fold process. Firstly, each message is digitally **signed** using **RSA 4096** encryption algorithm to ensure its authenticity. Secondly, the message is encrypted using end-to-end encryption (**E2E**) techniques **prior to being transmitted**.

The signed and encrypted message is designed to be accessible **exclusively by the intended recipient**. In the event of any unauthorized intervention, such as a man-in-the-middle attack aimed at manipulating the message content, the integrity of the communication chain is protected through the detection of a **signature violation**. This violation promptly invalidates the entire communication sequence.

Within this context, DEMOS serves as an **integrity and identity verification mechanism**. It operates with optimized resource utilization, solely employing the necessary computational power to certify messages. Importantly, DEMOS does not retain or decode the message contents in any manner. Consequently, this approach facilitates a **login-free** messaging service, oriented towards **user freedom**, that operates effectively on-chain.

On-chain Storage

Design and Rationale

On-chain storage, an infrequently observed phenomenon within the blockchain realm, is inherently scarce due to valid justifications. The decentralized, replicated, and persistent nature of the blockchain necessitates a highly resource-intensive procedure for storing files, demanding significant temporal, spatial, and computational resources.

Nonetheless, certain protocols have managed to construct relatively successful blockchains that facilitate file storage by adopting precise and targeted designs to address the aforementioned challenges. While some have achieved a degree of success, the majority suffer from deficiencies in robustness, scalability, or performance.

Within the DEMOS network, file storage is supported through leveraging the decentralized, agile, and flexible structure of nodes, employing various optimizations tailored to resolve three fundamental problems:

1. **The Size Dilemma:** This paramount issue confronts every storage service, entailing efficient management of dynamically sized files and optimal storage techniques.
2. **Hashing and Replication:** Unique to web3 processes, this facet encompasses ensuring resilience, security, and redundancy, while simultaneously maintaining network agility to ensure proper functioning.
3. **Block Inclusion:** As a blockchain-based service, the network must furnish adequate data within each block to validate and certify storage operations, while avoiding exponential growth in blockchain size beyond sustainable limits.

The Size Dilemma

The DEMOS On-chain Storage (OCS) protocol addresses this challenge by establishing an agreement between the client and the RPC node regarding the optimal chunk size for dividing files before transmission. This approach significantly enhances the speed and efficiency of transfers while maintaining robustness and error handling capabilities. By **dynamically optimizing** the workload of the protocol, varying file sizes are managed in a tailored manner.

Moreover, **chunking** plays a crucial role in data replication across a shard of nodes to prevent congestion within the internal protocol propagation.

In order to provide storage space, the DEMOS network assigns a **predetermined disk space** to each node as a prerequisite. This enables the even distribution of data propagation by determining the remaining space within the theoretically allocated storage capacity of the entire network.

Hashing and Replication

To achieve **full on-chain file service capability**, the OCS (On-Chain Storage) system employs a highly efficient mechanism for data routing and verification.

Upon uploading a file to the DEMOS, the designated RPC endpoint receives the file's individual chunks and computes their hashes as a means of ensuring **data integrity**. The client securely transmits the chunks via a signed communication stream, guaranteeing the **authenticity** of the request. Consequently, the RPC endpoint possesses a hashed and signed stream of the file.

At this juncture, a shard comprising a group of peer nodes is elected to **store the replicated version** of the file. Once the file stream has been replicated across a sufficient number of nodes, the remaining peers are notified with a reference to the specific nodes capable of retrieving the complete data. This approach ensures that while replication is robust and reliable, it **avoids unnecessary duplication** of the file stream on every individual node, thereby conserving network bandwidth and disk space. Nonetheless, each node can access the data **consistently** and with **redundancy**.

Block Inclusion

Incorporating complete file streams as contiguous units may initially appear to be a resilient concept, but in practice, it significantly **hampers performance** and **fails to yield tangible advantages**. Nevertheless, it is imperative for every block to formally record and publicly disclose the pertinent details concerning the stored data.

To enhance the efficiency of this process while upholding security and maintaining the integrity of the blockchain, the DEMOS network incorporates **within each block** the cryptographic hashes and the most recent replication map of the stored files. These elements are dynamically updated as necessary, ensuring the current status of the replication map is **accurately represented**.

Consequently, each block encompasses a comprehensive set of information, facilitating the **retrieval** of the current state of the uploaded files and providing insights into their behavior over time.